

DIT — DAKAR INSTITUTE OF TECHNOLOGIES

MASTER 2 — INGÉNIERIE DES DONNÉES / BIG DATA

Module : Architectures Data Lakehouse & Ingestion de données

EXAMEN — TRAVAIL À DOMICILE

Conception et construction complète d'une plateforme Data Lakehouse —
ingestion via Apache NiFi depuis des API publiques vers MinIO

ÉPREUVE INDIVIDUELLE — aucun travail en binôme ou en groupe n'est autorisé

Rendu : vidéo de soutenance de 15 min + code source complet — aucune base de code n'est fournie

Durée accordée : 4 semaines à compter de la diffusion du sujet

Enseignant responsable

M. PENE

Data Engineer & Data Lecturer — DIT (Dakar Institute of Technologies)

Nature de l'épreuve

Ceci est un examen individuel à réaliser à domicile (take-home exam), noté, et non un simple exercice d'application. Contrairement à un TP guidé, aucun projet de départ, aucun docker-compose et aucun code (DAG Airflow, jobs Spark) ne vous sont fournis.

- Vous devez concevoir et construire l'intégralité de la plateforme, du stockage à l'orchestration, en partant de zéro (« from scratch »), à partir uniquement du cahier des charges fonctionnel et technique ci-après.
- Ce sujet décrit des exigences et des contraintes à satisfaire, pas une architecture prête à copier. Les choix d'implémentation (structure du docker-compose, organisation du code, structure des buckets, schémas de données...) sont de votre responsabilité et seront évalués comme tels.
- L'évaluation porte autant sur le résultat fonctionnel que sur la compréhension démontrée à l'oral (dans la vidéo) de ce que vous avez construit.

Sommaire

1. Contexte et exigences de la plateforme cible

Vous devez concevoir et déployer, de bout en bout, une plateforme data lakehouse conteneurisée (Docker Compose) suivant une architecture en médaillon (Bronze → Silver → Gold), alimentée par une ingestion Apache NiFi depuis des API publiques, et interrogeable via un moteur SQL.

1.1 Pile technique imposée

Les technologies suivantes sont imposées ; leur agencement précis (versions compatibles, configuration réseau, volumes, ordre de démarrage) est à votre charge.

Couche	Technologie imposée	Rôle attendu
Ingestion	Apache NiFi	Interroger des API publiques et déposer les données brutes (JSON) dans MinIO
Orchestration	Apache Airflow	Déclencher et enchaîner les traitements Spark du médaillon
Traitement	Apache Spark (cluster : 1 master + au moins 1 worker)	Construire les couches Bronze, Silver et Gold en Iceberg
Format de table	Apache Iceberg	Format de table transactionnel (time travel, schema evolution)
Catalogue	Project Nessie	Catalogue versionné des tables Iceberg
Stockage objet	MinIO	Stockage S3-compatible : zone brute + warehouse Iceberg
Moteur de requête	Dremio OSS (imposé, pas de substitut)	Requêtage SQL des tables Bronze/Silver/Gold, à tester explicitement
Métadonnées	PostgreSQL	Base de métadonnées Airflow / Nessie
Supervision	Prometheus + Grafana	Monitoring de l'infrastructure et, si possible, de NiFi

1.2 Modèle de données cible

La plateforme doit traiter au moins deux domaines fonctionnels distincts issus de FakeStoreAPI, par exemple utilisateurs et produits (le domaine commandes, via les paniers, est valorisé — voir section 4). Pour chaque domaine, vous devez faire vivre le médaillon complet (Bronze, Silver, Gold) ; la définition précise de chaque couche est laissée à votre conception (voir Partie 4).

2. Objectifs pédagogiques de l'épreuve

Cette épreuve évalue votre capacité à :

- concevoir seul, de bout en bout, une architecture data lakehouse cohérente à partir d'un cahier des charges fonctionnel, sans code de départ ;
- déployer et faire communiquer, via Docker Compose, l'ensemble des briques imposées (NiFi, Airflow, Spark, MinIO, Nessie, Dremio, monitoring) ;
- concevoir un flux NiFi permettant d'ingérer des données depuis une API HTTP vers un stockage objet S3-compatible ;
- écrire vous-même les traitements Spark réalisant le médaillon complet (Bronze → Silver → Gold) en Iceberg ;

- orchestrer l'ensemble avec Airflow, du déclenchement de l'ingestion jusqu'à la disponibilité des données Gold ;
- documenter, argumenter et défendre oralement vos choix d'architecture.

3. Contraintes à respecter — l'architecture détaillée est à concevoir par vous

Important

Ce sujet ne fournit ni schéma d'architecture détaillé, ni code de départ : c'est un cahier des charges. La conception complète (organisation des buckets, format de stockage intermédiaire, fréquence de collecte, gestion des schémas, orchestration précise, gestion des erreurs, idempotence...) relève de vos choix, à documenter et à justifier dans votre vidéo de soutenance.

3.1 Contraintes obligatoires

1. Les données sources doivent provenir de l'API FakeStoreAPI (voir section 4). Aucun fichier CSV statique ne doit être utilisé comme source de départ.
2. Apache NiFi doit être l'unique point d'ingestion : il interroge l'API, effectue un contrôle minimal, puis dépose les données dans MinIO, dans une zone brute clairement séparée du bucket warehouse Iceberg. Le nom et la structure de cette zone brute sont à votre charge.
3. NiFi ne doit réaliser aucune transformation métier (jointure, calcul, agrégat) : uniquement collecte, contrôle et dépôt.
4. Vous devez écrire vous-même l'intégralité des traitements Spark du médaillon (lecture JSON depuis la zone brute, écriture Bronze, puis Silver, puis Gold, en tables Iceberg via le catalogue Nessie).
5. L'ensemble du pipeline doit être orchestré par un DAG Airflow que vous concevez, du déclenchement de l'ingestion (ou de la détection de nouvelles données) jusqu'à la mise à jour des tables Gold.
6. Le pipeline complet doit être démontrable de bout en bout : appel API par NiFi → dépôt dans MinIO → traitement Spark orchestré par Airflow → tables Gold interrogeables en SQL (Dremio ou équivalent).
7. Vous devez couvrir au moins deux domaines de données distincts issus de FakeStoreAPI (ex. produits et utilisateurs, éventuellement paniers).
8. Votre lakehouse doit constituer un historique reconstitué d'environ 6 mois de données, malgré l'absence de dimension temporelle native dans l'API (voir section 4.1).
9. Dremio doit être déployé et configuré comme moteur de requête SQL du lakehouse (connecté au catalogue Nessie et à MinIO). Son bon fonctionnement doit être testé explicitement : requêtes SQL réelles sur les tables Bronze, Silver et Gold, avec au moins une jointure entre domaines et une agrégation, exécutées et montrées en direct.

4. Source de données imposée : FakeStoreAPI

La source de données de cet examen est imposée : l'API publique FakeStoreAPI (<https://fakestoreapi.com>). Elle expose notamment les ressources suivantes, exploitables pour couvrir plusieurs domaines fonctionnels :

- /products (et /products/categories) — domaine « produits »
- /users — domaine « clients / utilisateurs »
- /carts — paniers, utilisables comme proxy du domaine « commandes »

4.1 Contrainte spécifique : reconstituer un historique de 6 mois

FakeStoreAPI ne renvoie que l'état courant des données, sans dimension temporelle. Votre plateforme doit malgré tout permettre de constituer, dans le lakehouse, un historique reconstitué d'environ 6 mois de données ingérées depuis cette API.

La façon de satisfaire cette contrainte (échelonnement des exécutions d'ingestion dans le temps, horodatage systématique de chaque exécution, backfill simulé avec des dates réparties sur 6 mois, snapshots périodiques, etc.) est entièrement à votre charge. Elle doit être conçue, mise en œuvre, puis expliquée et justifiée dans votre vidéo, et doit permettre des analyses temporelles en couche Gold (évolution dans le temps, tendances).

5. Travail demandé

L'épreuve est structurée en 7 parties. Les parties 1 à 6 sont obligatoires ; la partie 7 est un bonus. Tout doit être construit par vous, de zéro.

Partie 1 — Conception et déploiement de l'infrastructure complète (obligatoire)

1. Rédiger un fichier docker-compose.yml regroupant l'ensemble des services de la pile imposée (section 1.1) : MinIO, PostgreSQL, cluster Spark, Nessie, Dremio, Airflow, Prometheus, Grafana, et Apache NiFi.
2. Configurer le réseau Docker, les volumes persistants nécessaires et les ports exposés, en évitant les conflits.
3. Faire démarrer l'ensemble des services et vérifier leur bon fonctionnement (interfaces web accessibles, connectivité inter-services).
4. Créer dans MinIO les buckets nécessaires (zone brute et warehouse Iceberg) et configurer Nessie comme catalogue pour Spark et Dremio.

Partie 2 — Déploiement et configuration d'Apache NiFi (obligatoire)

1. Intégrer NiFi au réseau Docker de la plateforme afin qu'il puisse résoudre le nom du service minio.
2. Exposer l'interface web de NiFi et documenter les identifiants de connexion (génération automatique ou configuration simplifiée, à justifier).
3. Prévoir un volume persistant pour l'état interne de NiFi (repository de flowfiles, provenance, etc.).

Point d'attention technique

Apache NiFi récent nécessite HTTPS et une authentification par défaut (utilisateur généré automatiquement, mot de passe dans les logs du conteneur). Montrez dans votre vidéo la procédure de récupération de ces identifiants (docker logs) ou la configuration d'un accès HTTP simplifié, en la justifiant.

Partie 3 — Flux d'ingestion NiFi depuis FakeStoreAPI vers MinIO (obligatoire)

Concevez, dans NiFi, un dataflow qui interroge FakeStoreAPI et dépose les données JSON obtenues dans la zone brute de MinIO. Aucun processeur n'est imposé ni suggéré : le choix, l'agencement et la configuration des processeurs NiFi relèvent entièrement de votre conception. Vous devrez expliquer et justifier vos choix dans votre vidéo (pourquoi ces processeurs, comment est gérée la planification des appels, comment sont gérées les erreurs, etc.).

Seul point technique à respecter pour dialoguer avec MinIO : l'endpoint S3 doit pointer vers le nom du service Docker (ex. `http://minio:9000`, pas localhost), avec les identifiants MinIO que vous aurez définis. Le nommage du bucket et la structure des clés d'objet sont également à votre discrétion.

Partie 4 — Pipeline Spark du médaillon complet (obligatoire)

Développez vous-même les jobs Spark nécessaires pour construire, à partir de la zone brute alimentée par NiFi, les trois couches du médaillon en tables Iceberg (catalogue Nessie) : Bronze, Silver, Gold. Aucune règle de nettoyage, de jointure ou de calcul n'est imposée ni suggérée : la définition précise de ce que contient chaque couche, les règles de transformation Bronze → Silver → Gold et les indicateurs retenus en Gold sont entièrement à votre conception, en cohérence avec les données FakeStoreAPI et l'historique de 6 mois constitué. Ces choix devront être expliqués et justifiés dans votre vidéo.

Partie 5 — Test et validation du moteur de requête Dremio (obligatoire)

Dremio doit être configuré comme moteur de requête SQL du lakehouse, connecté au catalogue Nessie (source Iceberg) et à MinIO. Ce n'est pas une simple option de confort : son bon fonctionnement doit être testé et démontré explicitement.

1. Connecter Dremio au catalogue Nessie et vérifier que les tables Bronze, Silver et Gold de chaque domaine y sont visibles.
2. Exécuter et montrer en direct au moins une requête SQL sur une table Bronze, une sur une table Silver et une sur une table Gold, avec des résultats cohérents.
3. Exécuter et montrer une requête réalisant une jointure entre deux domaines (ex. produits et clients) directement depuis Dremio.
4. Exécuter et montrer une requête d'agrégation (ex. comptage, moyenne, group by) sur une table Gold.

Vous pouvez également créer, si vous le souhaitez, des vues (virtual datasets) Dremio au-dessus des tables Gold pour faciliter ces requêtes ; ce n'est pas obligatoire mais valorisé.

Partie 6 — Orchestration Airflow et démonstration de bout en bout (obligatoire)

Concevez un DAG Airflow orchestrant l'ensemble du pipeline. Deux approches de déclenchement de la partie Spark sont possibles, à choisir et justifier :

- **Option A — Déclenchement planifié** : le DAG vérifie périodiquement la zone brute de MinIO alimentée par NiFi et lance les jobs Spark dès que de nouvelles données y sont détectées.
- **Option B — Déclenchement événementiel** : NiFi appelle l'API REST d'Airflow (ou dépose un signal) à la fin de l'ingestion pour déclencher automatiquement le DAG.

Vous devez démontrer, en direct dans votre vidéo, le passage complet d'un appel API réalisé par NiFi jusqu'à sa visibilité dans une table Gold interrogeable via Dremio.

Partie 7 — Bonus (facultatif, points supplémentaires)

- Superviser NiFi via Prometheus/Grafana (reporting task Prometheus de NiFi) et ajouter un panneau au dashboard existant.
- Gérer les doublons ou les données déjà ingérées (idempotence) à l'aide d'un DistributedMapCache ou d'une vérification d'attribut.
- Couvrir le troisième domaine « commandes » (réel ou synthétique) en plus des deux domaines minimaux.
- Mettre en place un Process Group NiFi paramétrable (Parameter Context) pour rendre le flow réutilisable avec une autre API.
- Démontrer une fonctionnalité Iceberg avancée (time travel, schema evolution) sur au moins une table, interrogée depuis Dremio.

6. Livrables attendus

Le livrable principal de cette épreuve est une vidéo de soutenance individuelle. Aucun rapport écrit long n'est exigé : c'est la vidéo qui doit convaincre de la compréhension du travail réalisé et de votre capacité à en rendre compte seul(e).

6.1 Livrable principal — vidéo de soutenance de 15 minutes

Vous devez produire une vidéo individuelle de 15 minutes environ (14 à 17 minutes, voix commentée, écran partagé, votre visage n'est pas requis) qui parcourt l'ensemble de la plateforme et du code que vous avez produits. La vidéo doit couvrir, dans l'ordre :

1. Présentation de l'architecture que vous avez conçue (schéma à l'écran) et des choix structurants (1-2 min).
2. Revue du docker-compose.yml complet : tous les services, réseau, volumes, ports (1-2 min).
3. Démonstration du flow NiFi dans l'interface graphique : parcours des processeurs, configuration vers MinIO, choix de routage et de gestion des erreurs (3 min).
4. Présentation des jobs Spark du médaillon (Bronze/Silver/Gold) : structure du code, principaux traitements (2-3 min).
5. Test du moteur de requête Dremio : requêtes SQL sur Bronze/Silver/Gold, une jointure entre domaines, une agrégation (2-3 min).
6. Démonstration en direct de bout en bout : déclenchement du DAG Airflow, appel API par NiFi, dépôt dans MinIO, exécution Spark, requête finale dans Dremio (2-3 min).
7. Bilan : difficultés rencontrées, choix techniques justifiés, éventuel bonus réalisé (1 min).

La vidéo est déposée sur une plateforme accessible à l'enseignant (lien non listé YouTube/Drive, ou fichier déposé sur l'espace de rendu du cours) — le lien ou le fichier doit être fonctionnel jusqu'à la correction.

6.2 Livrables complémentaires (obligatoires mais non notés séparément de la vidéo)

- L'intégralité du code source produit (docker-compose.yml, jobs Spark, DAG Airflow, export du flow NiFi au format .json), déposé sur un dépôt Git (GitHub/GitLab) privé partagé avec l'enseignant, ou archive .zip.
- Ces éléments servent de preuve et pourront être ouverts par l'enseignant pendant ou après le visionnage de la vidéo ; ils ne remplacent pas la vidéo, qui reste le support d'évaluation principal.

Consignes pour la vidéo

Montrez l'écran réel (terminal, navigateur, interfaces NiFi/Airflow/Dremio/MinIO), pas des diapositives seules.

Le respect du format 15 minutes fait partie de la notation : une vidéo trop courte (incomplète) ou beaucoup trop longue (non synthétique) sera pénalisée.

Vous devez être seul(e) à l'écran et à l'oral : c'est une épreuve individuelle, la vidéo fait office de soutenance.

7. Modalités et barème indicatif

Délai de rendu : 4 semaines à compter de la diffusion du présent sujet. Passé ce délai, le rendu (vidéo + code source) ne sera plus accepté sauf dérogation accordée par l'enseignant.

Critère	Points
Partie 1 — Infrastructure complète déployée et opérationnelle	3 pts
Partie 2 — NiFi déployé et intégré au réseau de la plateforme	2 pts

Critère	Points
Partie 3 — Flux NiFi d'ingestion depuis une API vers MinIO, correct et robuste	4 pts
Partie 4 — Pipeline Spark médaillon complet (Bronze/Silver/Gold), écrit intégralement par l'étudiant	4 pts
Partie 5 — Test et validation du moteur de requête Dremio (requêtes, jointure, agrégation)	3 pts
Partie 6 — Orchestration Airflow et démonstration de bout en bout	3 pts
Qualité de la soutenance vidéo (clarté, respect des 15 min, compréhension démontrée)	1 pt
Partie 7 — Bonus	+2 pts

Total : 20 points + 2 points de bonus.

Épreuve individuelle — rappel

Cet examen est un travail strictement individuel. Toute similarité anormale entre deux rendus (code ou vidéo) sera considérée comme une fraude et fera l'objet d'une procédure disciplinaire.

L'utilisation d'assistants IA pour comprendre des concepts, déboguer ou apprendre est tolérée en amont, mais vous devez être capable d'expliquer et de justifier chaque ligne de votre plateforme pendant la vidéo de soutenance. Une incapacité à expliquer un choix technique lors du visionnage pourra entraîner une pénalité, voire une convocation pour un oral complémentaire.

8. Ressources utiles

- Documentation Apache NiFi : <https://nifi.apache.org/docs.html>
- Processeur InvokeHTTP : documentation officielle pour interroger une API HTTP/REST depuis NiFi.
- Processeur PutS3Object : documentation officielle des propriétés S3 (endpoint override, path style access) — à consulter pour la compatibilité avec MinIO.
- Documentation MinIO — compatibilité S3 : <https://min.io/docs/minio/linux/reference/s3-api-compatibility.html>
- Documentation Apache Airflow : <https://airflow.apache.org/docs/>
- Documentation Apache Iceberg : <https://iceberg.apache.org/docs/latest/>
- Documentation Project Nessie : <https://projectnessie.org/>
- FakeStoreAPI (source de données imposée) : <https://fakestoreapi.com>

8.1 Dépôts de référence (modules précédents)

Les dépôts suivants, issus de travaux précédents du module, peuvent être consultés pour vous familiariser avec les briques technologiques (Spark, ingestion, structuration d'un lakehouse). Ils ne constituent ni une architecture à reproduire telle quelle, ni une base de code de départ pour cet examen : l'intégralité de votre plateforme et de votre flux NiFi doit rester votre propre conception, conformément à la section 3.

- <https://github.com/PENE18/modern-data-platform>
- https://github.com/PENE18/DIT_spark-etl-exercice_application
- https://github.com/PENE18/DIT_TP_INGESTION